School of Computing and Information Systems

COMP30023: Computer Systems

Practical Week 1

1 Introduction

Git is a distributed version control system originally created by Linus Torvalds to maintain Linux kernel code. It is now widely used in all kinds of projects (both open-source and commercial), to track changes in files and enable collaboration among developers.

This lab will serve as a brief introduction to Git so that you can start using it to manage code in your own projects.

1.1 Register for GitHub

Projects will be submitted to our GitHub Organisation.
Complete the GitHub Access assignment (for project submissions) before the end of Week 3.

If you don't already have an account, create a GitHub account at `https://github.com`.

1.2 Create a key pair

Complete `VM Access` assignment Prelab Part 1.
Complete Prelab Part 2 before the next practical.

2 Install git

Refer to the Installing Git page of the Git book to install `git` on your local machine.

After installation, run `$ git --version` in your terminal to verify that `git` has been installed correctly.

3 Introducing Yourself to Git

Before we start using Git, we need to setup our name and email. Note that this is just for identification purposes, and not to log in to any system. This process will need to be repeated on each user account on which you want to use git.

Command: `$ git config --global user.name "Your Name"`

Command: `$ git config --global user.email "yourusername@student.unimelb.edu.au"` [1]

Now that Git knows who we are, let's create our first repository.

4 Creating a Git Project

Git projects are more commonly known as Git *repositories*. Let's initialise a repository.

1. Create a new directory. Let's call it `prac1`.

   Command: `$ mkdir prac1`

[1] The `--global` option means that Git will use these values for every Git project the current user creates. The values are written to `~/.gitconfig`, or `~/.config/git/config`.

2. Navigate into the newly created directory.
   Command: `$ cd prac1`

3. Initialise a repository in this directory. Command: `$ git init --initial-branch=main`

   If you get the error `unknown option `initial-branch=main`` , you have an older version of `git`.
   Run `$ git init` now and `$ git branch -M main` after making your first commit (Section 6).

   This will create a new (hidden)[2] directory named `.git` in the `prac1` directory. This directory will contain all version information related to this repository.

   Note: Do not initialise a `git` repository at `~`.
   If you did this accidentally, use `$ cd ~ && mv .git/ git.bak`

Sample solutions will be released at the end of the week.
Ask your classmates or tutor for help during practical classes.

5 Adding Files to Git

A file can exist in three stages in a Git repository: *Working, Staging and Commit*[3]

- Any changes not explicitly added to the Staging area reside in the Working Directory.
- Changes can be added to the Staging Area (also known as Index) using the `git add` command. This marks the changes that should go into the next commit.
- The `git commit` command adds the marked changes from the step above to the (local) repository.

Figure 1[4] shows the flow of operations:

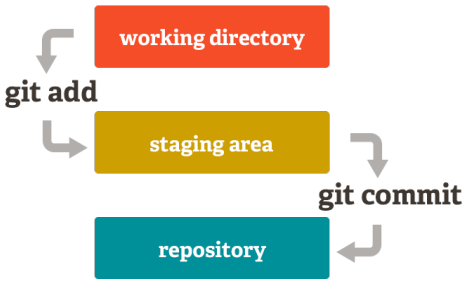


Figure 1: Flow of operations

Let's see this in practice:

1. Create an empty file named `test.c` with the `touch` command.
   Command: `$ touch test.c`

2. This file is now in the working directory and *untracked* by Git. Run the `git status` command to see this:
   Command: `$ git status`

3. Add some content to the file to make it compilable, using your preferred text editor:
   e.g. the line `void main(){}` in `test.c`.

   Brief instructions for `nano`:
   `$ nano test.c`, type in text, ctrl-x, letter y, enter key.

[2] You can use the command `ls -a` to view the directory.
[3] You can also stage parts of a file, which is a main reason for the intermediate staging step.
[4] Source: `https://git-scm.com/about/staging-area`

Brief instructions for `vim`:
`$ vim test.c`, letter i to enter insert mode, type in text, escape key to exit insert mode and back to normal mode, :x, Enter to save.

4. Add this file to the staging area.
   Command: `$ git add test.c`

5. View the status of the repository now using the `$ git status` command again. Notice that the `test.c` file is now listed under 'Changes to be committed'.

6. If you have modified multiple files and want to add all of them to the staging area at once, you can use the `$ git add .` command.

6 Committing Changes

Now that we have added our changes to the staging area (i.e. marked the changes we want to commit), let's commit the changes.

Committing the changes stores a snapshot of the current state of the files. This allows you to roll back to this version if needed in the future. Note that this snapshot is stored *locally* on your machine.

1. Command: `$ git commit -m "Created test.c with main()"`
   Every commit needs to be accompanied by a *commit message*, which is what we specified using the `-m` parameter.

2. Inspect the current status of the repository using `$ git status`.

7 `.gitignore`

Reproducible files and directories, such as build artifacts (e.g. executables compiled from source code) and packages (e.g. `node_modules`) should (generally) not be committed to `git` repositories.

Given that they might differ from one version of the code to the next, we will be bloating our repositories if we tracked them in `git`. When repositories are then pushed to servers, all of these rubbish files will have to be uploaded to the server!

While it's possible to selectively stage files and commit (e.g. `$ git add file1 file2`), the convenience of using `$ git add .` / `$ git add -A` is lost. This is the purpose of the `.gitignore` file, which can be used to specify (in separate lines) the files and directories which should be ignored by `git`.

Write a `Makefile` to compile `test.c` with `gcc` or `clang` to an executable named `test`.
Discuss with your classmates and/or tutor if you have forgotten how to do so.

If you have access to a C compiler and `make` on your current machine:

Let's compile `test.c` with our C compiler.

1. Execute `$ make`.

2. Verify that compilation was successful (i.e. does `test` exist? is `test` executable?).

If not:

Create a new file called `test`.
Let's pretend that this is a binary compiled from the source code `test.c`.

Joining together:

We have a file named `test` which we want to ignore.

1. Execute `$ git status` again. Both `Makefile` and `test` should be unstaged at this point.
2. Let's create a `.gitignore` file, containing a single line:
   `test`
   This tells git to ignore files named `test`.

   We can also use wildcards, e.g. `*.o` to ignore all `.o` files.
3. Check `$ git status` again. Once you're happy, stage `Makefile` and `.gitignore` simultaneously and commit the changes, giving the commit an appropriate message.
4. Use the `$ git log` command to view the history of commits you have made.

Note: You are expected to follow good version control practices (sensible commit messages, no useless executables etc.) for project repositories. More on this in the project specifications.

8 Setting up GitHub

It is common practice in teams to have a `git` server that everyone pushes their code to.
GitHub and GitLab are examples of publicly-available hosting services. We will be using GitHub.

1. Login to GitHub.
2. Click 'New Repository' to create a project named `comp30023-prac-1` under your GitHub user.
3. Set the visibility level to 'Private' and deselect all initialization options.

We will be using `SSH` to interact with GitHub, the `git` server.

1. Copy your public key from Prelab Part 1.
2. In GitHub Settings (right top of the webpage, click your profile image), select "SSH and GPG keys".
3. Select "New SSH key".
4. Paste the key in, and click "Add SSH key" button.

9 Pushing Changes

Note that every `git` operation up to now has been done *locally*. That is, our changes to the files have not been sent to a remote server.

1. For this, we need to tell Git which external repository (remote) we want to operate on.
   Think of a remote as a pointer to the location of an external repository.
   `$ git remote add origin git@github.com:<username>/comp30023-prac-1.git`
   Replace `<username>` with your GitHub username.

   You can check the URLs using `$ git remote -v` and where necessary, amend it using
   `$ git remote set-url origin <new-url>`.
2. Push your changes to the server by using the `git push` command.
   `$ git push -u origin main`

   After this first `push` you can push changes by simply using the command `$ git push`.

   Note: `-u` is short for `--set-upstream`, which sets an upstream branch for the local branch. In this scenario, the local branch `main` is associated with the branch named `main` on the `origin` remote.

   If you're using Windows with WSL, and have created your key pair in the WSL environment, please ensure that you use `git` under WSL also.

It is appropriate to `push` after every major change (e.g. a new feature, a bug fix, some refactoring etc.).

10 Cloning

It is common to join a team with an ongoing project which does source code management using Git. Imagine that you are joining a team that has been working on a project for a while. You will need to get a copy of the codebase to start working on it. We use the `git clone` command to get a copy of existing code stored on an external Git server.

Use your GitHub username in the subsequent step.

Joining together:

1. Run this command from a directory **outside** the Git repository (`prac1`) we have been working in:

   ```
   $ git clone git@github.com:<username>/comp30023-prac-1.git prac1-partner
   ```

2. Note that a new directory has been created with the name `prac1-partner`. Browse to this directory, and view the files. Use the `$ git status` and `$ git log` commands to note that the full history of the repository has been downloaded.

11 Branches

Git allows you to organise code in branches. For example, you might create a new branch for a new feature of your application. You can then keep continued development on the core of the application (e.g. bug fixes) separate from the new feature development.

In this exercise, we will explore the concept of branches and the merge workflow, assuming 2 independent users working with 1 remote repository.

You are going to play the role of 2 users (you and a fake partner).

Open 2 windows, and navigate to both `prac1` and `prac1-partner` directories. You will own the folder `prac1` and your fake partner will own `prac1-partner`.

11.1 Creating and Switching Branches

1. You are yourself and under the directory `prac1`.

2. When initialising the repository, we specified the default branch to be named '`main`'. Use the `git branch` command to list the current branches.

   Command: `$ git branch`

   The currently active branch is shown with an asterisk.

3. When working with others, you'd typically want the pull to get latest changes before branching.
   Command: `$ git pull`

4. The most common way to create a new branch in Git is to create a new branch and switch to it in the same command.

   Let's create a new branch named '`feature-me`'.

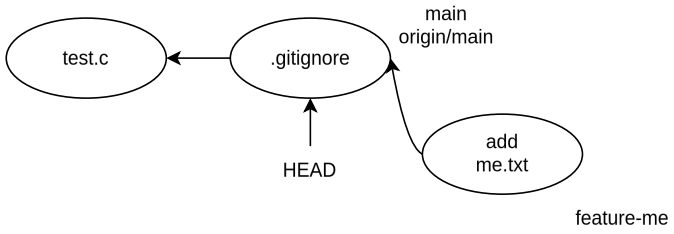
   Command: `$ git checkout -b feature-me`, or `$ git switch -c feature-me`.

5. Run the `git branch` command again. Notice the newly created branch is currently active.

6. Create a new file named `me.txt`, put your name in the txt file, then add it to the staging area and commit it.

7. Switch back to the `main` branch.
   Command: `$ git checkout main` or `$ git switch main`

   If we were to depict the current state of our repository graphically, it might look something like:



Note that `HEAD` (the reference to where we are in the repository) is currently a reference to the `main` branch, since that's the branch that we have asked `git` to switch to.

8. List the files (including hidden ones) in the directory using the `ls` command. Notice that we only see `.gitignore`, `Makefile`, `test` and `test.c`. This is because the change we made was in the other branch named '`feature-me`'.

11.2 Pushing and Pulling branches for review (and to save progress)

In a collaborative workflow, users will typically push their branches to the remote repository for review before merging them to a shared development branch (such as `main`).

The merge process can be achieved purely through `git`, or Pull Requests/Merge Requests, which are extended features provided by typical `git` server software.

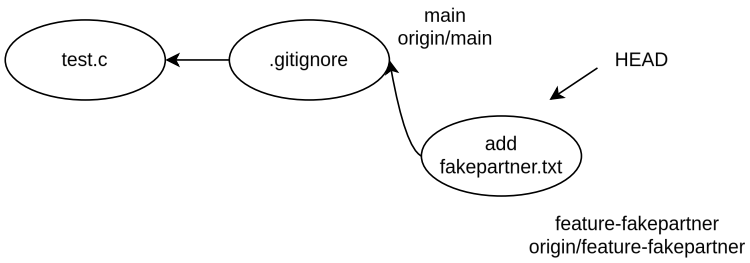
In this section, your fake partner (you pretend) will push their branch and you will review it:

1. You are pretending to be your fake partner.
   Navigate to folder `prac1-partner`

2. Fake partner decides to create their feature branch using:
   `$ git checkout -b feature-fakepartner`

3. Fake partner creates a new file named `fakepartner.txt`, puts their `fake name` in the file, then adds it to the staging area and commits it.

4. Fake partner pushes their feature branch to the remote:
   `$ git push -u origin feature-fakepartner`

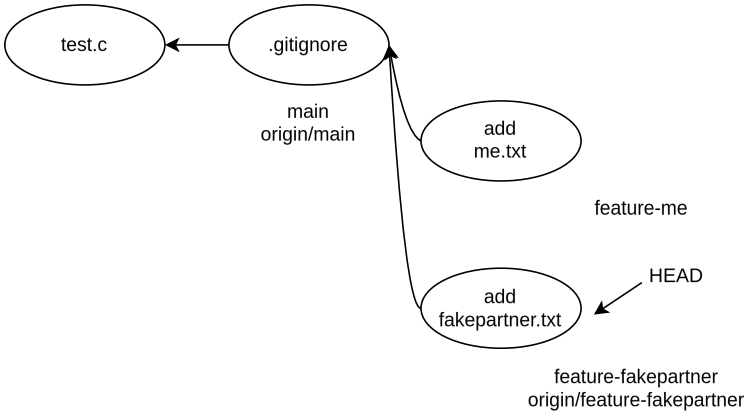


5. After the fake partner has pushed their branch, you (stop pretending, be yourself) navigate to `prac1`:
   `$ cd ../prac1`

   Then fetch the remote repository:
   `$ git fetch`

   Behind the scenes, `git` will 'download' the partner's commits, and create a remote tracking branch, referencing the latest commit which they have pushed to their branch.

6. You can now switch to your fake partner's branch and take a look at the files in the directory:
   `$ git switch feature-fakepartner`



You should be able to see the file that they've created.

7. To review, you may want to compare their branch with the `main` branch:
   `$ git diff main`, or `$ git diff main..feature-fakepartner`.

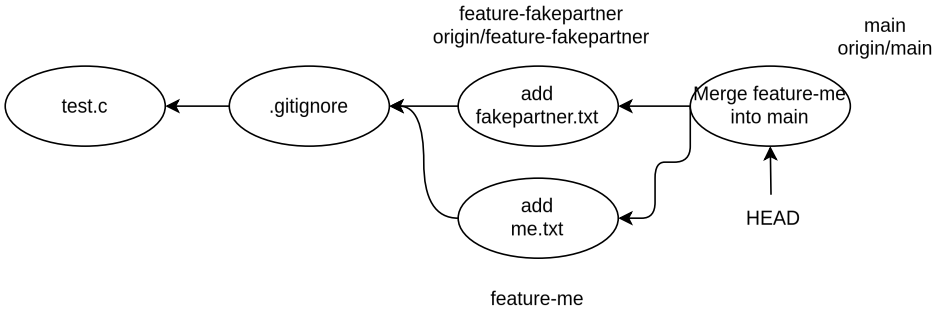
11.3 Merging

When collaborating with others, the merging of branches can get quite tricky.

Try to follow this scenario:

1. Pretend to be the fake partner and navigate to the `prac1-partner` directory.
2. Fake partner finishes their work, receives approval to merge, and merges their branch to `main`:
   `$ git checkout main`
   `$ git merge feature-fakepartner`
   This is a 'fast-forward' merge, since `git` can directly 'append' the commits which have been made on `feature-fakepartner` to the latest commit on `main`[5].
3. Fake partner pushes the updated `main` branch to the remote repository.
4. Stop pretending to be the fake partner, and navigate to the `prac1` directory.
5. You switch to `main` and pull the `main` branch, receiving the fake partner's changes from the remote repository.
6. You have now also finished work, and merges your `feature-me` branch to `main`:
   Given that a fast-forward merge is no longer possible, `git` will now prompt you for a commit message, and create a merge commit containing changes from both (parent) branches, `main` and `feature-me`.
   In this scenario, `git` was able to automatically perform the merge as the changes were made in separate files.
7. You push the `main` branch.
8. Pretend to be the fake partner and navigate to the `prac1-partner` directory.
9. Fake partner pulls the `main` branch.
   Now, both you and your fake partner should be able to see both of the created files.

[5] In reality, there's no append operation performed. `git` will simply update the reference of `main` (a commit hash pointing to the latest commit on `main`) to the commit hash of the latest commit on `feature-fakepartner`.

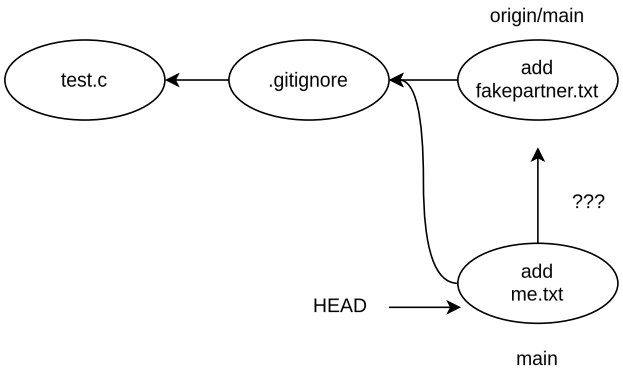


Now, what happens if you didn't pull the changes from your fake partner (Step 5) before merging their branch?

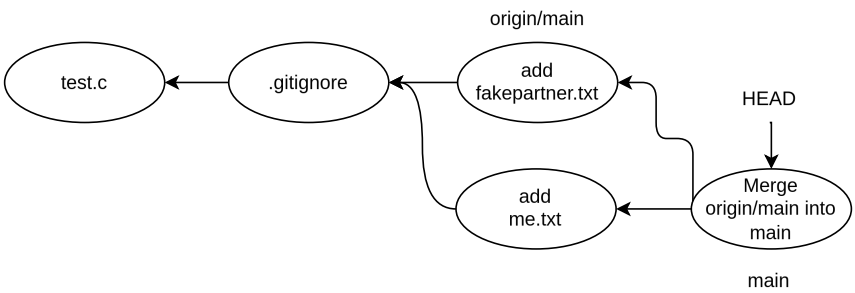
In this scenario, the remote server will reject the push if you attempt to push the `main` branch.

```
$ git push
To github.com:<username>/comp30023-prac-1.git
! [rejected]        main -> main (non-fast-forward)
error: failed to push some refs to 'github.com:<username>/comp30023-prac-1.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

This is because `git` doesn't know which commit to keep. If it allowed your push, then the fake partner's commits will be lost (at least from the perspective of the remote `main` branch)!



In this case, one of the commands that you can run to resolve this is `$ git pull`, which will (by default) create a merge commit between the remote tracking `origin/main` branch and local `main` branch.



After the merge commit has been created, you will be able to push `main` to the remote repository.

11.4 Branch deletion

After merging, we can delete our feature branches and push the deletion to the remote repository:

1. Under the `prac1` directory, delete the `feature-me` branch:
   `$ git branch -d feature-me`

   As we have not previously pushed this branch, it should not exist in the remote repository.

2. Navigate to the `prac1-partner` directory and pretend to be the fake partner.

3. Delete the `feature-fakepartner` branch locally and push the deletion to the remote repository by:
   `$ git branch -d feature-fakepartner`

   `$ git push origin :feature-fakepartner`

4. On the remote repository (GitHub), the `feature-fakepartner` branch should now be gone.

5. Navigate back to the `prac1` directory and perform a `$ git fetch`.

   Note that the branches `main` and `feature-fakepartner` should be present, because remote deletions do not affect the state of local branches.

6. Delete the `feature-fakepartner` branch.

11.5 Merge conflicts (Exercise)

Another common occurrence when using `git` collaboratively is merge conflicts. A merge conflict occurs when `git` encounters changesets which can't be automatically merged. For examples, if two users changes the same line of code differently in 2 separate branches, `git` won't be able to automatically merge these branches together.

With minor difference, we will repeat the steps in Section 11.1 Creating and Switching Branches and Section 11.3 Merging.
This time:

1. You should create a file named `group.txt` with `ME: APPLE` as the content of the file.

2. Fake partner should also create a file named `group.txt`, but with `FAKE_PARTNER: BANANA` as the content of the file.

When the second merge happens, `git` will prompt the user to resolve merge conflicts, e.g.:

```
$ git merge feature-<branchname>
Auto-merging feature-<branchname>
CONFLICT (add/add): Merge conflict in group.txt
Automatic merge failed; fix conflicts and then commit the result.

$ git status
You have unmerged paths.
(fix conflicts and run "git commit")
(use "git merge --abort" to abort the merge)

Unmerged paths:
(use "git add <file>..." to mark resolution)
both added:      group.txt

$ cat group.txt
<<<<<<< HEAD
ME: APPLE
=======
FAKE_PARTNER: BANANA
>>>>>>> feature-<branchname>
```

In this case, we want to resolve the conflict such that `group.txt` has both names.
Open the file in your text editor and remove the merge markers, i.e.:

```
$ cat group.txt
ME: APPLE
FAKE_PARTNER: BANANA
```

And complete the merge commit with:
`$ git add group.txt`
`$ git commit`

After completing this exercise successfully, both you and your partner should have `group.txt` on the `main` branch, with 2 names in the file.

`git status` should report the state as `nothing to commit, working tree clean`.

12 Tags

Git is a Version Control System. It allows the labelling of different versions of code (e.g. like in software versions such as Linux 6.7).

You can add a tag to a commit using:
`$ git tag -a v0.15 -m "Release version 15"`

The `-a` parameter is the tag label or 'annotation'. The `-m` argument allows the addition of a message to the tag.

Tags in Git need to be explicitly pushed (by default).
You can push a single tag using `$ git push origin v0.15`, or all created tags using `$ git push --tags`.

13 Resources

Git is complex and feature-rich; we have only just scratched the surface of its capabilities in this short hour. The resources below will help you learn more.

- `https://www.youtube.com/watch?v=2sjqTHE0zok`
  MIT Missing Semester: Lecture 6.
- `https://www.atlassian.com/git/tutorials/merging-vs-rebasing`
  Merge vs Rebase Workflows
- `https://git-scm.com/book/en/v2`
  A canonical reference. Authoritative and extremely well written.
- `https://jvns.ca/blog/2024/02/16/popular-git-config-options/`
  Popular `git config` options
- `https://wizardzines.com/git-cheat-sheet.pdf`
  git cheat sheet
- `http://eagain.net/articles/git-for-computer-scientists/`
  In the author's own words: 'Quick introduction to git internals for people who are not scared by words like Directed Acyclic Graph.'
- `http://gitolite.com/uses-of-index.html`
  Why the index/staging area is so useful

Git is very widely used, which means there is an abundance of resources on the Internet. Happy learning!

A Sample solutions

```
#!/bin/sh
set -e
set -x

# Install git
git --version

# Some preparaton
rm -rf wd && mkdir wd && cd wd

# 4. Creating a Git Project
mkdir prac1
cd prac1
git init --initial-branch=main

# 5. Adding files
touch test.c
git status
echo "void main() {}" > test.c
git add test.c
git status || true

# 6. Committing Changes
git commit -m "Created test.c with main()"
git status

# 7: .gitignore
echo -e "test: test.c
\tcc -o test test.c" > Makefile
make
./test || true
file test && stat test

git status || true
echo "test" > .gitignore
git status || true
git add .
git commit -m "added .gitignore"
git log

# 8. Setting up 'GitHub'
#######################################
# For the purpose of making it easier to
# test following steps, mock the remote
# repository (create a dir locally which
# we pretend is our remote repository)
cd ..
mkdir remote/
git --git-dir=remote/ init
#######################################

# 9. Pushing changes
cd prac1/
#######################################
# This is the mocked path
git remote add origin ../remote
#######################################
git push -u origin main

# 10. Cloning
cd ..
#######################################
# This is the mocked path
git clone -b main remote/ prac1-partner
cd prac1-partner && git log && cd -
#######################################

# 11. Branches
# 11.1. Creating and switching branches
# You create commit on feature-me
cd prac1/
git branch
git pull
git checkout -b feature-me
git branch
touch "me.txt"
echo "my name" > me.txt
git add -A
git commit -m "add me.txt"
git checkout main
ls -la
cd -

# 11.2. Pushing and Pulling branches
# Partner commit on feature-partner, push
cd prac1-partner/
git checkout -b feature-fakepartner
touch "fakepartner.txt"
echo "fake name" > fakepartner.txt
git add -A
git commit -m "add fakepartner.txt"
git push -u origin feature-fakepartner
cd -

# You review
cd prac1/
git fetch
git switch feature-fakepartner
git diff main..feature-fakepartner
cd -

# 11.3. Merging
# Partner merges to main
cd prac1-partner/
git checkout main
git merge feature-fakepartner
git push
cd -

# You
cd prac1/
git checkout main
git pull
git merge feature-me --no-edit
git push
ls
cd -

# You
cd prac1-partner/
git pull
ls
```

```
cd -

#######################################
# Extended scenario

# Revert repositories/remote to
# previous state
# Typically, you shouldn't rewrite
# history like this, especially so
# when working with others

# The use of these commands is outside
# scope of prac1
git --git-dir=remote/ update-ref \
    refs/heads/main refs/heads/main~2

# Reset user repos
cd prac1/
git fetch --all
git reset origin/main --hard

cd ../prac1-partner/
git fetch --all
git reset origin/main --hard
cd ../
#######################################

# Partner merges to main
cd prac1-partner/
git switch main
git merge feature-fakepartner
git push
cd -

# You merge without pulling first
cd prac1/
git checkout main
git merge feature-me
git push || true

# You pull to create merge commit
git pull --no-ff --no-edit
git push
cd -

# Partner pulls
cd prac1-partner/
git pull
cd -

#######################################

# 11.4. Branch deletion
# You
cd prac1/
git branch -d feature-me
cd -

# Fake Partner
cd prac1-partner/
git branch -d feature-fakepartner
git push origin :feature-fakepartner
cd -

# You
cd prac1/
git fetch
git branch -d feature-fakepartner
cd -

# 11.5. Merge conflicts
# For brevity, demonstrating with 1 repository
cd prac1/

# Create branch named feature-fakepartner from main
git branch feature-fakepartner

# You add to main your change
git checkout main
echo "ME: APPLE" > group.txt
git add -A
git commit -m "group.txt ME"

# Fake partner makes their change
git checkout feature-fakepartner
echo "FAKE_PARTNER: BANANA" > group.txt
git add -A
git commit -m "group.txt fake partner"

# Fake partner merges
git checkout main
git merge feature-fakepartner || true # Merge conflict
cat group.txt

# Remove markers, keep both lines (with ': ')
sed -i -n '/: /p' group.txt
git add -A
git commit --no-edit
cat group.txt
```